

# Networking Fundamentals: TCP, UDP, IP

*Part I · Foundations — how machines actually talk to each other.*

## LEARNING OBJECTIVES

- Explain why the network is the lifeline — and the weak point — of every distributed system.
- Describe the layered model of networking and what “encapsulation” means.
- Explain what **IP** does, what an IP address and a packet are, and what IP does *not* guarantee.
- Explain what a **port** is and how one machine runs many services at once.
- Describe how **TCP** gives reliable, ordered delivery — including the three-way handshake — and what it costs.
- Describe how **UDP** gives fast, connectionless delivery, and when that trade is worth it.
- Turn networking realities (latency, unreliability, limited bandwidth) into concrete design rules.

## REAL-WORLD ANALOGY — THE POSTAL SYSTEM

Sending data across the internet is like sending mail. Every house has an **address** (an IP address). A big shipment is split into many numbered **boxes** (packets) that may travel different roads and arrive out of order. You can send it two ways: **registered mail** that confirms delivery and re-sends anything lost, arriving complete and in order (that’s **TCP**) — or a cheap stack of **postcards** you simply drop in the box and never track, fast but with no guarantees (that’s **UDP**). And just as a letter is placed in an envelope, then a mailbag, then a truck, your data is wrapped layer by layer as it travels. Hold this picture; the whole chapter is inside it.

## 3.1 Why networking is the heart of system design

In Chapter 2 we saw that a single machine has hard limits, so real systems use many machines. The moment you have more than one, they must communicate — and every one of those conversations crosses a **network**. Remember the bottom of the memory hierarchy: a network round trip is thousands of times slower than a memory read, and across the world it can take ~150 milliseconds. Worse, unlike your local RAM, the network is **unreliable**: messages can be lost, delayed, duplicated, or arrive out of order.

### WHAT PROBLEM DOES THIS CHAPTER SOLVE?

Every remote call your system makes — app to database, service to service, user to server — is a bet placed on an unreliable, sometimes slow network. If you don’t understand how that network behaves, your “working” design will mysteriously hang,

drop data, or crumble under load. This chapter gives you the mental model to reason about all of it.

## 3.2 Networking is built in layers

The internet is too complex to design as one giant blob, so it is split into **layers**, each responsible for one job and each relying on the layer below it. A simplified, practical view has four layers:

| Layer                | Job                                                              | Example                 |
|----------------------|------------------------------------------------------------------|-------------------------|
| <b>Application</b>   | The actual thing you want to do.                                 | HTTP, DNS, email        |
| <b>Transport</b>     | Get data between two <i>programs</i> , reliably or fast.         | TCP, UDP                |
| <b>Internet (IP)</b> | Get a packet from one <i>machine</i> to another across networks. | IP (addresses, routing) |
| <b>Link</b>          | Move raw bits over an actual wire or radio to the next device.   | Ethernet, Wi-Fi         |

(You may also hear of the **OSI 7-layer model** — a more detailed academic version of the same idea. The four layers above are enough for system design.)

### Encapsulation: wrapping data layer by layer

As your data goes down the layers to be sent, each layer **wraps** it with its own header (a small block of control information) — exactly like putting a letter in an envelope, then a mailbag, then a truck. The receiver unwraps them in reverse. This wrapping is called **encapsulation**.

SENDING (data goes DOWN the layers, each adds a header):

```

Application:           [ HTTP request: GET /home ]
Transport (TCP):       [ TCP hdr | HTTP request ]   + adds PORTS, seq #s
Internet (IP):         [ IP hdr | TCP hdr | HTTP request ] + adds IP ADDRESSES
Link (Ethernet): [ Eth hdr | IP | TCP | HTTP | Eth end ] + sent as raw bits
  
```

RECEIVING: the far machine unwraps each layer in reverse order.

### KEY INSIGHT — WHY LAYERS MATTER TO YOU

Layering means each layer can change without breaking the others. Wi-Fi vs Ethernet (Link) doesn't affect HTTP (Application). TCP vs UDP (Transport) doesn't change IP (Internet). As a designer you mostly work at the Application and Transport layers — choosing HTTP, gRPC, TCP, or UDP — and trust the layers below to carry your bits.

### 3.3 IP — addressing and delivery (best effort only)

**IP (Internet Protocol)** is the layer that gives every machine an **address** and moves small chunks of data, called **packets**, from one machine toward another.

- An **IP address** identifies a machine on the network. **IPv4** looks like `142.250.72.14` (about 4 billion possible addresses — now running out). **IPv6** is the newer, vastly larger scheme (e.g. `2607:f8b0:4005::200e`).
- Data is broken into **packets**, each stamped with a source and destination IP address.
- Packets travel through many **routers** (the internet's intersections). Each router just reads the destination and forwards the packet one **hop** closer — no single router knows the whole path.

```
your PC --> home router --> ISP --> [ router ] --> [ router ] --> server
                        each "hop" reads the destination IP and forwards onward.
No router knows the full route; each just passes it one step closer.
```

#### CRUCIAL — IP PROMISES ALMOST NOTHING

IP is **“best effort”** only. It does *not* guarantee that a packet arrives, that packets arrive *in order*, or that they arrive only *once*. Packets can be dropped when a router is overloaded, take different routes, or show up out of sequence. IP just tries its best. Any reliability you need must be added by the layer above — and that is exactly the problem TCP solves.

### 3.4 Ports — many programs, one machine

An IP address finds the right *machine*, but a server runs many programs at once — a web server, a database, an SSH login. How does an arriving packet know which one it's for? The answer is a **port**: a number (0–65535) that identifies a specific program on the machine.

The pair (**IP address + port**) is called a **socket** and uniquely identifies one endpoint of a connection. Some ports are standardized (“well-known ports”):

| Port      | Service            | Port       | Service            |
|-----------|--------------------|------------|--------------------|
| <b>80</b> | HTTP (web)         | <b>443</b> | HTTPS (secure web) |
| <b>22</b> | SSH (secure login) | <b>53</b>  | DNS (name lookup)  |

So `142.250.72.14:443` means “the HTTPS program on that machine.” Ports are why one server can host dozens of services on a single IP.

### 3.5 TCP — reliable, ordered delivery

**TCP (Transmission Control Protocol)** sits on top of IP and turns its unreliable packet delivery into a **reliable, ordered stream** of data. If IP is a pile of postcards that might get lost, TCP is registered mail that guarantees everything arrives, complete and in order.

## What problem does TCP solve?

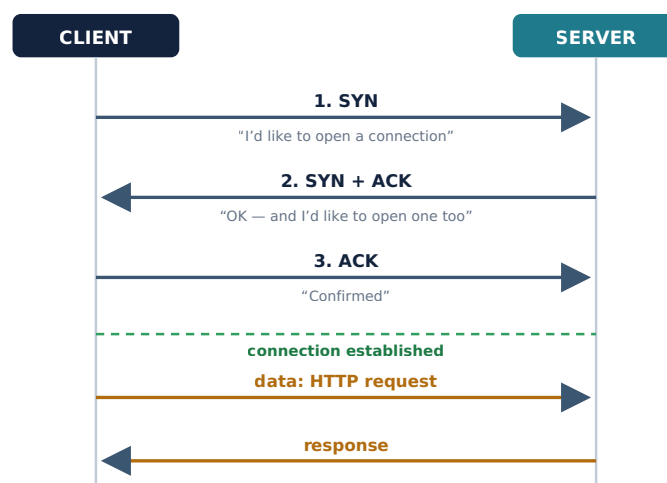
IP packets can be lost, duplicated, or reordered. Most applications — loading a web page, downloading a file, a database query — cannot tolerate missing or scrambled data. TCP fixes all of that so the application can pretend the network is a clean, ordered pipe.

## How TCP works

- **Connection-oriented:** before sending data, the two sides establish a connection with a **three-way handshake** (below). This confirms both can send and receive.
- **Sequence numbers:** every chunk of data is numbered, so the receiver can reassemble them *in order* even if packets arrive scrambled.
- **Acknowledgements & retransmission:** the receiver confirms (ACKs) what it got. If the sender gets no ACK in time, it **resends** — this is how lost packets are recovered.
- **Flow control:** the receiver tells the sender how fast it can accept data, so a fast sender doesn't overwhelm a slow receiver.
- **Congestion control:** TCP senses network congestion and slows down to avoid making a traffic jam worse.

## The three-way handshake

Before any real data flows, TCP opens the connection in three steps — think of it as a quick “Can you hear me? — Yes, can you hear me? — Yes.”



Notice: a full round trip is spent **before any real data is sent**. That setup cost is why reusing connections matters (Section 3.8).

### WHAT TCP COSTS

All these guarantees aren't free. The handshake adds a **round trip of latency before any data moves**. Tracking sequence numbers and ACKs adds overhead. And TCP's strict ordering causes **head-of-line blocking**: if one early packet is lost, everything behind it must wait for the resend, even if it already arrived. For most apps this is a great trade — but not for all, which is why UDP exists.

## 3.6 UDP — fast and connectionless

**UDP (User Datagram Protocol)** is the opposite philosophy: send data immediately, with no connection, no ordering, and no guarantee of delivery. It's the stack of postcards — you just drop them and hope.

### What problem does UDP solve?

TCP's reliability adds latency (handshakes, waiting for resends). For some applications, **being fast matters more than being perfect**, and a little lost data is acceptable or better handled by the app itself. UDP strips away the overhead to deliver the lowest possible latency.

- **Connectionless:** no handshake — the first packet is real data. No setup delay.
- **No guarantees:** packets may be lost, reordered, or duplicated, and UDP won't fix it. The application decides what to do.
- **Lightweight & fast:** tiny headers, no tracking — ideal for high-speed, real-time data.

#### WHEN LOSING DATA IS ACTUALLY FINE

In a live video or voice call, a packet that arrives late is *useless* — the moment has passed. It's better to skip it and keep playing than to freeze the whole call waiting for a resend (which is exactly what TCP would do). That's why real-time media, online games, and **DNS lookups** use UDP: speed and freshness beat perfect delivery.

## 3.7 TCP vs UDP — the decision

| Property        | TCP                                                                              | UDP                                                                        |
|-----------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Reliability     | Guaranteed delivery (resends lost data)                                          | No guarantee                                                               |
| Ordering        | In order                                                                         | May arrive out of order                                                    |
| Connection      | Handshake first (connection-oriented)                                            | None (connectionless)                                                      |
| Speed / latency | Slower to start; more overhead                                                   | Fast; minimal overhead                                                     |
| Best for        | Web, APIs, file transfer, databases — anything that must be correct and complete | Live video/voice, gaming, DNS, streaming telemetry — speed over perfection |

#### A MODERN TWIST

The newest web protocol, **HTTP/3**, actually runs over UDP (via a protocol called QUIC) and rebuilds reliability on top — getting UDP's low latency *and* TCP-like guarantees, while avoiding head-of-line blocking. It's a good reminder that these are building blocks you can combine, not a rigid either/or.

## 3.8 Networking realities that shape every design

Beginning engineers quietly assume the network is fast, reliable, and free. It is none of these. These assumptions are so common and so wrong they have a name — the **fallacies**

**of distributed computing** (covered fully in Chapter 19). Three matter most here, each with a direct design consequence:

| The reality                                                                               | The design rule it forces                                                                                              |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>The network is not reliable</b> — calls can fail or hang.                              | Always set <b>timeouts</b> ; add <b>retries</b> for transient failures; expect and handle failure everywhere.          |
| <b>Latency is not zero</b> — every round trip costs real time, especially across regions. | <b>Minimize round trips</b> ; batch requests; put servers and data <b>near users</b> (CDNs, regional replicas).        |
| <b>Setting up a TCP connection costs a round trip.</b>                                    | <b>Reuse connections</b> — use connection pooling and HTTP keep-alive instead of opening a new connection per request. |

#### KEY INSIGHT — CONNECTION REUSE IS ONE OF THE CHEAPEST BIG WINS

If every request opens a fresh TCP connection (plus a TLS handshake for HTTPS), you pay one or more round trips of pure setup *before* any useful work. Multiply that across millions of requests and it's enormous. **Connection pooling** (keeping a set of open connections ready to reuse) and **keep-alive** (reusing one connection for many requests) remove that cost. This is why databases and HTTP clients pool connections by default — and why creating a new client per request is a classic performance bug.

#### PITFALL — A TIMEOUT DOES NOT MEAN IT FAILED

When a request times out, you often *cannot tell* whether the server never got it or did the work and only the reply was lost. Blindly retrying a non-idempotent action (like “charge the card”) can do it **twice**. This is why safe retries require **idempotency** — designing operations so that doing them twice is harmless (Chapter 37). Networking's unreliability reaches all the way up into how you design your APIs.

Two more realities worth knowing now: **bandwidth is limited and costs money** (moving lots of data between regions or out to users is a real line on the cloud bill), and network gear like **firewalls and NAT** can block or rewrite connections. Later, **load balancers** (Chapter 7) will make a choice you can now understand: operate at **Layer 4** (just forward TCP packets, fast and simple) or **Layer 7** (understand HTTP and route by URL, smarter but heavier).

## Common mistakes

#### WATCH OUT

- Assuming the network is reliable — not setting timeouts, not handling failed or hung calls.
- Opening a new TCP/TLS connection per request instead of pooling or keeping connections alive.
- Designing “chatty” systems that make many sequential round trips where one batched call would do.
- Choosing TCP when UDP fits (real-time media) or UDP when you actually need reliability.
- Treating “connection established” or “request sent” as if it meant “delivered and processed.”
- Blindly retrying a non-idempotent request after a timeout — risking doing it twice.

- Assuming bandwidth is free and infinite, then being surprised by cross-region data-transfer bills.

## Interview questions

1. Walk through the TCP three-way handshake. Why is it necessary, and what does it cost?
2. Compare TCP and UDP. Give two real applications for each and justify the choice.
3. What does IP guarantee, and what does it explicitly *not* guarantee?
4. What is a port, and why can one machine with one IP run many services?
5. Why does opening a new connection for every request hurt performance? How do you fix it?
6. What is head-of-line blocking in TCP, and how does HTTP/3 avoid it?
7. After a request times out, is it safe to retry it? What does the answer depend on?
8. Name three “fallacies of distributed computing” and a design consequence of each.

## Hands-on exercises

1. Run `ping` to a nearby site and a distant one; record the round-trip times and explain the difference using distance.
2. Run `tracert` (or `tracert`) to a website and count the hops. Note that no single router knew the whole path.
3. For each of these, decide TCP or UDP and justify: web browsing, a video call, a DNS lookup, downloading a zip file, a live multiplayer game.
4. If one connection setup costs 1 round trip and the round-trip time is 80 ms, calculate the setup time wasted by opening 50 separate connections versus reusing 1.
5. Look up and write down the well-known ports for HTTP, HTTPS, SSH, and DNS.

## Mini project

### TRACE A REAL REQUEST’S JOURNEY

Open your browser’s developer tools, go to the **Network** tab, and load any website. Click the main request and read its **timing breakdown**: you’ll see separate slices for DNS lookup, initial connection (the TCP handshake), TLS setup, and finally content download. Note how much of the total time is *setup* versus actual data transfer. (If you prefer the command line, run `curl -w` with a timing format string against a URL.) Seeing setup cost with your own eyes makes the case for connection reuse unforgettable.

## Large project (capstone continues)

### YOUR SOCIAL PLATFORM — THE NETWORK LAYER

In your design notebook, add a short “**network assumptions**” section. Decide: which interactions are simple request/response over **HTTP/TCP** (loading a profile), which need a **persistent connection** for real-time push (live notifications — foreshadowing

WebSockets in Chapter 28), and whether anything could use **UDP** (e.g. live video). Mark where you'll rely on **connection pooling**, where **cross-region latency** will hurt, and which operations must be **idempotent** so they can be safely retried when the network drops a response. You'll reuse these decisions throughout Parts II and V.

## Chapter summary

- Networking is organized in **layers** (Application, Transport, Internet/IP, Link); each layer wraps the one above with a header — **encapsulation**.
- **IP** addresses machines and moves **packets** hop-by-hop through routers, but is **best-effort only**: packets may be lost, reordered, or duplicated.
- **Ports** identify which program on a machine a packet is for; an **IP + port** is a socket.
- **TCP** turns IP's unreliable packets into a **reliable, ordered stream** using a handshake, sequence numbers, acknowledgements, and retransmission — at the cost of setup latency and head-of-line blocking.
- **UDP** is **fast and connectionless** with no guarantees — ideal when speed beats perfection (live media, gaming, DNS).
- The network is **unreliable, latent, and bandwidth-limited**. That forces timeouts, retries, connection reuse, minimizing round trips, placing data near users, and designing for **idempotency**.

### COMING NEXT

**Chapter 4 — The Web: HTTP, HTTPS, DNS.** We'll build on TCP to see what happens when you type a web address: how **DNS** turns a name into an IP, how **HTTP** structures requests and responses, and how **HTTPS** adds encryption on top — completing the journey of a single web request.